FilsPay

TeamName

Member	Owned
Member 1	Owner: ...
Member 2	Owner: ...
Member 3	Owner: ...

Live: fp-pay-tzakir.dilmune.app · fp-admin-tzakir.dilmune.app · fp-api-tzakir.dilmune.app Code: github.com/thrzkr/fils-pay

2 · Root cause

We trusted generated code, comments and tests instead of the ledger.

Tests were written to assert the bugs and "Claude says" comments closed questions without a check. So four patterns repeated everywhere:

Pattern	Symptoms it explains
Check-then-act, no lock	Refunds could pay out twice · duplicate payout webhooks · payment links double-charge · sweep posts twice
Two sources of truth	Dashboard 245 bps vs billing 225 · rebuild button ≠ CLI · webhook signed ≠ sent · **payout ≠ ledger after refunds**
Money not integer fils end to end	CSV adjustments 10x too small · float32 payouts · int32 volume counter wraps at 2.1M BD
Auth / tenancy checked in some places only	OTP optional · login lands in another merchant · refund another merchant's payment · impersonation can move money

3 · Key findings, ranked by impact

Top four

#	Finding	Evidence	Status
1	**The login code was optional.** The password-step cookie already worked on every merchant endpoint.	Pending cookie → merchant API returned data; test now gets 401.	**Fixed**
2	**One merchant's password opened another merchant.** Session created for any user with that email.	Shared-email login test (owner@alpharetail is in 2 merchants).	**Fixed**
3	**Payouts ignore refunds.** Merchant paid for money already returned to cardholders.	Demo day: batch paid 31.769 BD; ledger payable −17.107 BD; reconciliation gap ≠ 0.	**Open (finance)**
4	**Finance CSV adjustments 10x too small.** 4.75 BD posted as 475 fils; tests expected it.	Import test fails on old parser, passes on parseBhdToFils.	**Fixed**

Also fixed: concurrent refunds double-paying, duplicate payout webhooks (incident 08-21), webhook signatures, API key rotation, int32/float32 money, and a dashboard crash after every refund (found this week).

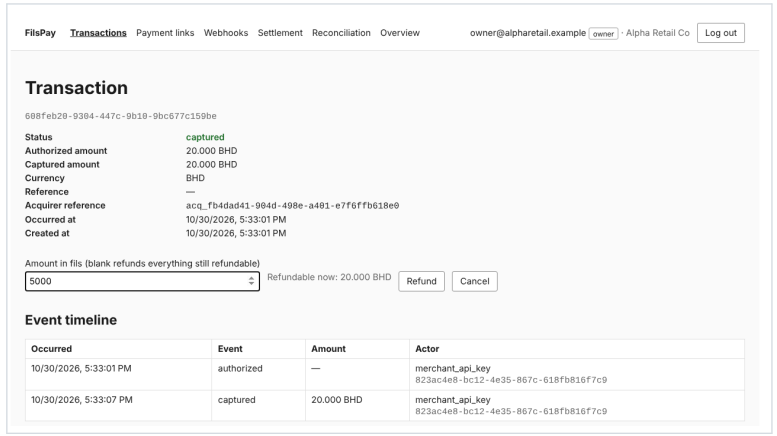
Every fix has a test that fails on the old code, or a query / live request

Fix	Proof
Sessions require authenticated = true; session tied to the checked (merchant, email)	`app.test.ts`: pending cookie 401; cross-merchant login rejected
Refunds: FOR UPDATE + pending reservation (0073), captured − refunded − pending, fee reversal	`refunds.test.ts` parallel refunds; demo fee reversals 281 + 112 fils
One settlement.paid per batch (0076, ON CONFLICT)	`payout_notify_test.go`: 8 concurrent enqueuers → 1 event
Fee display uses billing's effective_from rule	Old code restored: 58 pass / 5 fail → fixed: 377 / 0
BIGINT/int64 running totals (0078); integer Go payouts; largest-remainder splits	Go tests above 2^31 and 2^24
Webhooks sign the bytes they send; callbacks deduped (0074); link reservation (0075)	Load test: 0 double payments at 150 buyers
Dashboard bhd() accepts signed amounts (refund events)	bhd(−5000): 8 pass / 1 fail before, 9/9 after
API site → reverse proxy; HTTPS; systemd; setup.sh fails loudly	Secret files 404; /healthz /readyz 200; ports 3000/8091 closed

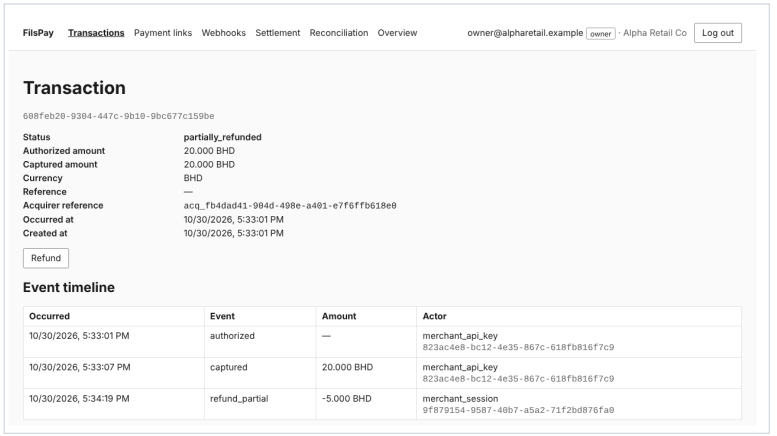
```
Fresh Postgres 16, 30 Oct:  72 migrations apply · npm test 453/453 · go test ./... all packages ok
```

5 · Live demo: full and partial refund

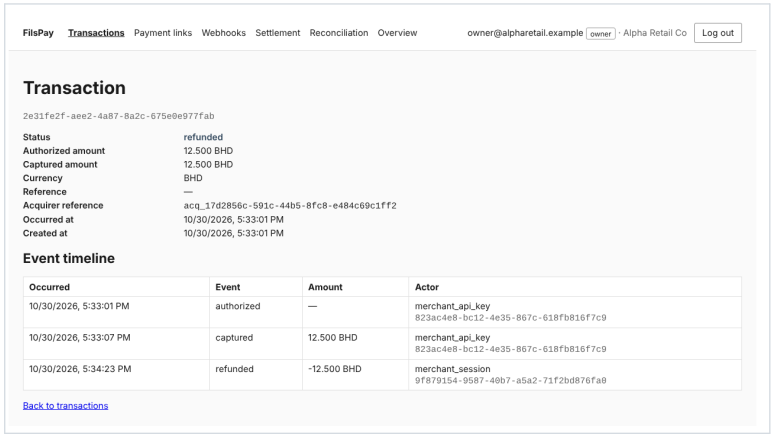
12.500 BD refunded in full · 20.000 BD refunded 5.000



Refund drawer: 5000 fils, refundable 20.000



partially_refunded · timeline −5.000 BHD

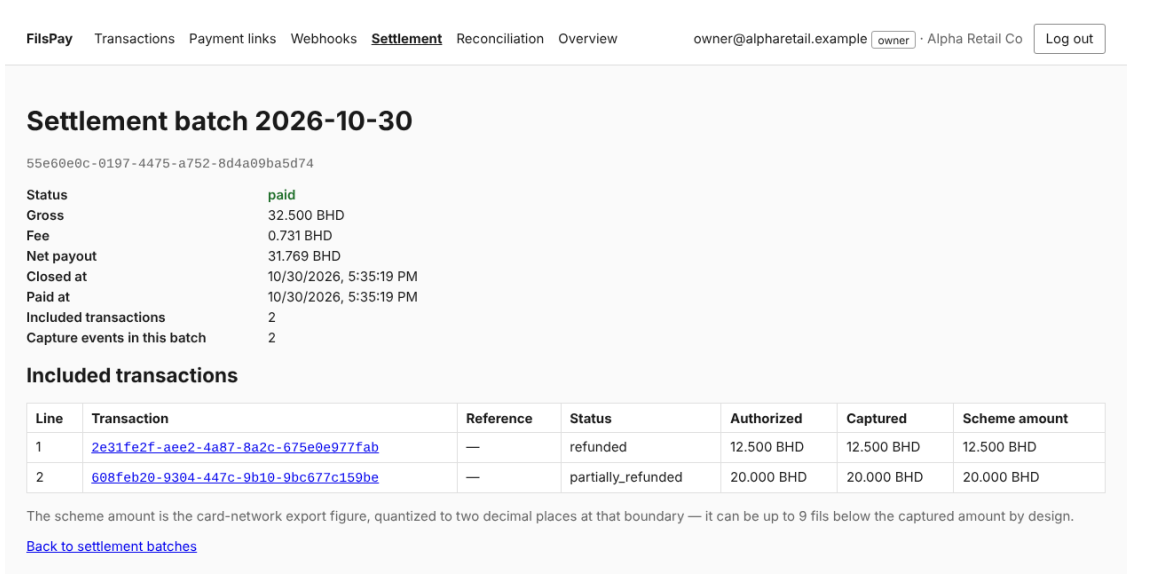


refunded · timeline −12.500 BHD

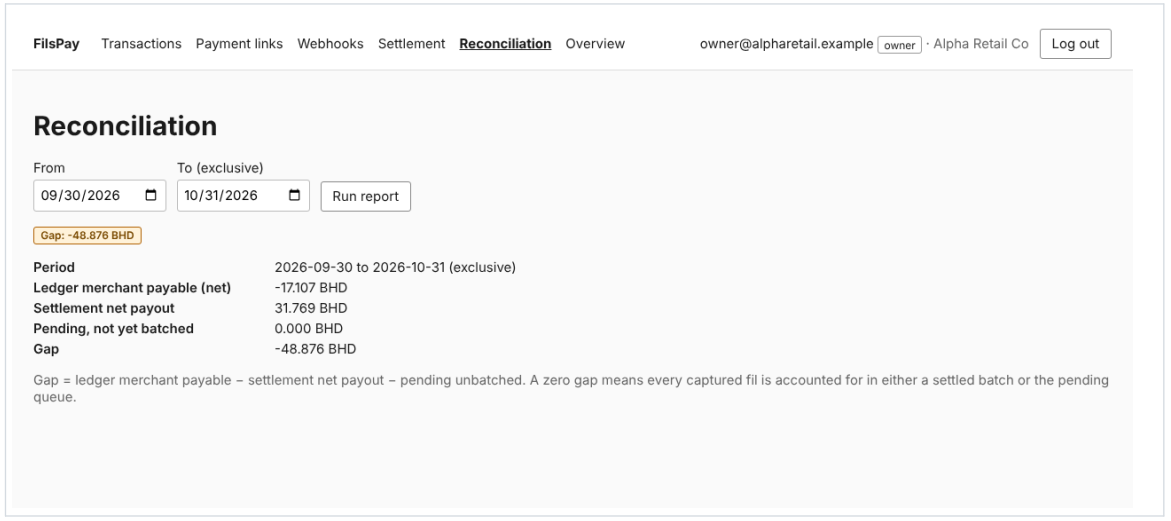
Fee reversed: 281 + 112 fils (2.25%, half-to-even). Settlement batch for the day: gross 32.500, fee 0.731, **net 31.769**: the refunds are in the ledger but not netted from the payout (finding 3, open).

5 · Live demo: settlement report

Backup screenshots if the live demo fails



Settlement batch 2026-10-30: both transactions, refunded / partially_refunded



Reconciliation: gap ≠ 0 shows payouts don't net refunds

6 · Under load

Estimate ~8 checkouts/s, ~10 concurrent buyers at p95 < 2 s; measured

Buyers	Checkouts/s	p95	Shed (503)	Double payments
10	8.2	1.65 s	0	0
20	8.1	3.15 s	0	0
40	7.9	5.71 s	0	0
150 (shed at 10)	7.6	1.89 s	4,111	0

Architecture. Cloudflare → nginx 443 → static fp-pay / fp-admin, API on 127.0.0.1:3000 (systemd, 0.5 vCPU / 512 MB), Go settlement (127.0.0.1:8091), reporting worker, Postgres 16. Redeploy: `bash ops/dilmune/setup.sh`.

Bottleneck: the API's 0.5 vCPU (throughput flat, memory ~100 MB). **Overload:** over 10 in-flight → 503 + Retry-After; retries are safe with the same Idempotency-Key; systemd restarts in ~2 s. We tried 40 first: p95 hit 5.7 s, so we cut it to 10.

Claude Code + CLAUDE.md rules + permission rules + /verify-fix, /check-live, /money-review

Rule: the AI proposes; only a failing-then-passing test, a query or a live request closes an item.

When it was wrong: we asked it to rotate the leaked admin password. It reported success and blamed the seed script when the login still failed. We asked for state, not output: a read-only query showed the password unchanged (`|f`), and the console history showed the reset had never run. It disproved its own seed theory (one admin row, no cron/timer/pm2). We only accepted `UPDATE 1 … |t`.

And this week: its own report said refunds worked "end to end". Driving the real dashboard with Playwright showed the page crashed after every refund, and that payouts ignore refunds.

What we chose not to change, and why

Decision / risk	Why	Safeguard
Payouts don't net refunds (not changed)	Finance owns net vs clawback vs hold; payout change on deadline day is riskier	Hold payouts where ledger payable < 0; alert on reconciliation gap ≠ 0
No backfills (CSV 10x, rotated keys, MDR rewrites)	Needs finance sign-off per file / merchant	Queries per finding, dry-run first
Secrets still in git history	History rewrite = force push, owner's call	Rotate admin, acquirer sandbox and DB passwords now
Postgres 5432 open	Dilmun dashboard change	Delete firewall rule #4; drop 0.0.0.0/0
Login code shown on screen (demo)	No email/SMS sender yet	Simulated money only; remove OTP_DEV_EXPOSE before real use

Still open: scheme residue posting (ADR-0002) · reaper for stuck pending refunds · OTP rate limits · Dilmun auto-deploy.

Appendix

Evidence

```
$ npm test            (fresh Postgres 16)
# pass 4 · 40 · 385 · 11 · 8 · 5  = 453, fail 0
$ go test ./...       ok config internalapi ledger money scheduler settlement splits

transaction | amount | captured | status             | refund | fee_reversed
2e31fe2f... |  12500 |    12500 | refunded           |  12500 |          281
608feb20... |  20000 |    20000 | partially_refunded |   5000 |          112
$ settlement run-batch --date=2026-10-30   gross=32500 fee=731 netPayout=31769
ledger merchant_payable: credits 32893, debits 50000 -> -17107
```

Full findings (26), root causes, decisions and screenshots: TeamName_FilsPay_Submission.pdf · `bugs.md` · `docs/AUDIT-FP-300.md`.